

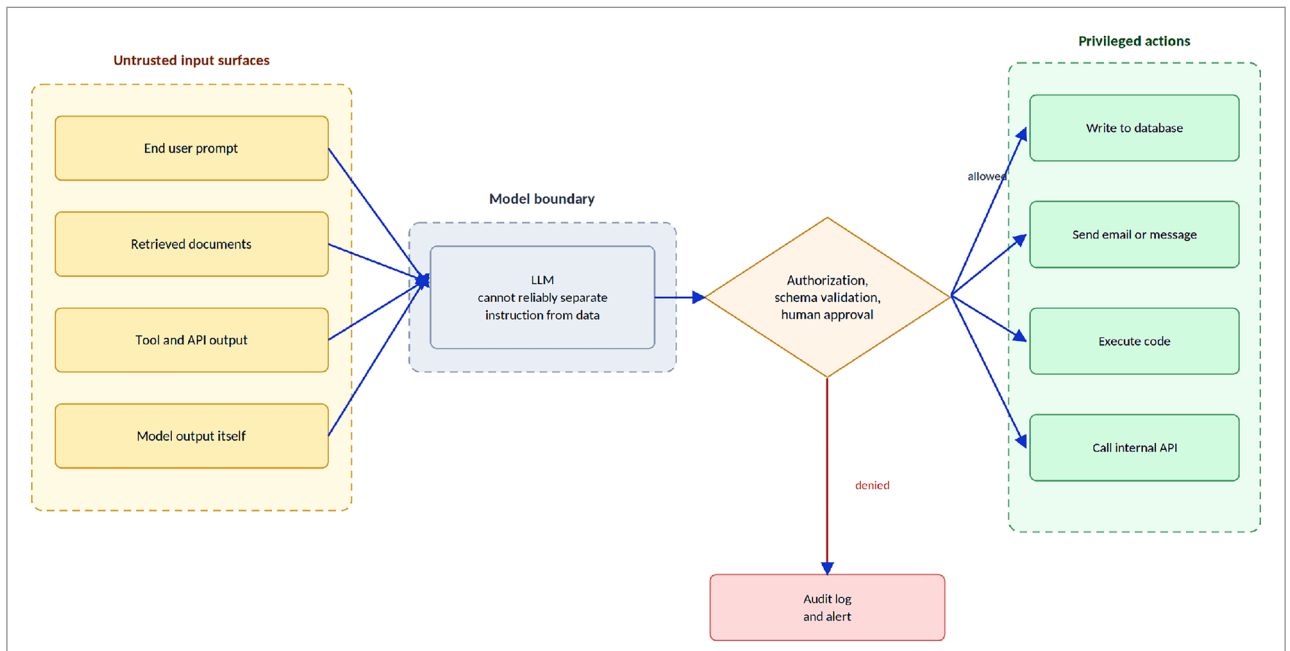


Figure 4: Trust boundaries. Retrieved documents and tool results are attacker-influenced input, and model output is untrusted until validated

The model supply chain is a binary supply chain

Model files are executable artifacts. Serialisation formats based on Python pickle can run arbitrary code at load time. This is not theoretical, and documented CVEs have covered remote code execution when loading untrusted models. Scanners have their own weaknesses, with bypass vulnerabilities in a widely used pickle scanner disclosed in late 2025.

Build an ingestion gate and make it the only path into production:

- Prefer formats that cannot execute code during deserialisation, such as safetensors, and refuse pickle-based artifacts unless there is a documented exception.
- Verify provenance cryptographically. Sigstore-backed model signing gives you a tamper-evident record of who produced the artifact.
- Scan artifacts and their dependency graph, then record the result as an attestation rather than a one-time check.
- Publish an AI bill of materials capturing model lineage, training data description, and evaluation results, using the ML-BOM extensions now present in the major SBOM formats.
- Mirror approved models into an internal registry and block direct pulls from public hubs in production networks.

The dependency side matters as much as the weights.

Package ecosystems around ML tooling have been targeted through dependency confusion and through package names hallucinated by coding assistants, which attackers then register.

Policy as Code at the admission boundary

Written standards do not stop a pod from starting. Admission policy does.

Two open source engines dominate. Both are worth understanding because they fit different teams.

Dimension	OPA with Gatekeeper	Kyverno
Policy language	Rego, general purpose	YAML, Kubernetes native
Learning curve	Steeper	Lower for Kubernetes teams
Scope	Kubernetes and beyond	Kubernetes focused
Mutation support	Available	First class
Best fit	Policy shared across systems	Cluster guardrails and defaults

A representative rule for an inference workload looks like this.

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: restrict-model-sources
spec:
  validationFailureAction: Enforce
  rules:
```